

GEOASSET

INTELLIGENCE PLATFORM

Pipeline Infrastructure Risk & Maintenance Management System

Final Project Report

Domain	Municipal water pipeline infrastructure risk and maintenance
Core stack	QGIS / PyQGIS, PostgreSQL / PostGIS, Python, FastAPI
Engineering	GeoPandas, Shapely, OSMnx, Pytest, Docker, GitHub Actions
Project CRS	EPSG:25832
Repository	github.com/AisunZa/geoasset-intelligence-platform

September 2026

Executive Summary

The GeoAsset Intelligence Platform is an end-to-end geospatial engineering project for identifying, visualizing, and serving pipeline infrastructure risk. The system combines realistic OpenStreetMap spatial context with synthetic pipeline and operational records, stores the data in PostgreSQL/PostGIS, calculates risk using spatial SQL, validates the results independently in Python, visualizes the outputs in QGIS, and exposes asset and risk information through a FastAPI REST/GeoJSON service.

The project is deliberately broader than a map-only GIS exercise. It demonstrates database design, geospatial ETL, spatial analysis, desktop GIS, PyQGIS automation, API development, automated testing, containerization, and continuous integration in one reproducible workflow. A custom GeoAsset Inspector plugin connects QGIS directly to the API, allowing an operator to search for a pipeline, review its risk factors, and zoom to its geometry.

Key outcome

The final network contains 500 pipeline segments totaling 54.34 km. The assessment identified 30 High/Critical assets. PIPE-00469 ranked highest with a risk score of 87.46 (Critical), 3 failures, 106 buildings within 100 m, and 79.81% flood exposure.

Project Profile

Pipeline segments	500
Network length	54.34 km
OSM buildings	9,865
Failure records	267
Inspection records	1,224
Maintenance records	318
High/Critical assets	30
Highest-risk asset	PIPE-00469 - 87.46 (Critical)

Report Structure

This report follows the full system lifecycle: problem definition, data, architecture, database design, ETL, risk methodology, QGIS analysis, the PyQGIS plugin, API delivery, testing/deployment, results, limitations, and conclusions.

1. Introduction

Infrastructure asset management depends on both spatial context and operational history. A pipeline may be old but isolated, or relatively young but located in a dense urban area with repeated failures and flood exposure. A useful prioritization system therefore needs to combine multiple risk dimensions rather than rely on a single attribute.

GeoAsset Intelligence was developed as a portfolio-grade demonstration of how a modern GIS workflow can support this type of decision-making. The project treats PostGIS as the analytical data core, Python as the ETL and validation layer, QGIS as the operational visualization environment, FastAPI as the delivery interface, and Docker/GitHub Actions as the reproducibility and quality layer.

2. Problem Definition and Objectives

The central problem is prioritization: given a network of pipeline assets, which segments should receive inspection or maintenance attention first? The platform addresses this question by combining five factors: asset age, inspection recency, failure history, nearby building exposure, and flood exposure.

The project objectives were to:

- Build a spatially enabled relational database for pipeline assets and operational records.
- Create a reproducible Python ETL workflow using real-world map context and synthetic utility data.
- Implement a transparent, weighted pipeline risk model in PostGIS.
- Independently validate risk calculations in Python/GeoPandas.
- Visualize the network and risk classes in QGIS.
- Extend QGIS with a lightweight PyQGIS asset-inspection plugin.
- Publish asset, risk, and GeoJSON results through a REST API.
- Add automated tests, containerization, and CI using a real PostGIS service.

Design principle

The same risk information should be consistent across the database, Python validation code, QGIS, the PyQGIS plugin, and the API. This makes the project a connected system rather than a collection of separate demos.

3. Data and Spatial Context

The project uses a hybrid data strategy. OpenStreetMap provides realistic road and building geometry, while the pipeline network and operational history are synthetic so the workflow can be reproduced without exposing utility data. Pipeline geometry is generated from road geometry, producing a network that follows plausible urban alignments while remaining clearly demonstrative.

Dataset	Records / role	Source type
Pipeline segments	500 segments; 54.34 km	Synthetic network from OSM road geometry
Buildings	9,865 features	OpenStreetMap
Failures	267 records	Synthetic operational data
Inspections	1,224 records	Synthetic operational data
Maintenance	318 records	Synthetic operational data
Risk zones	Flood exposure polygons	Demonstration spatial risk layer

All analytical layers are stored and processed in EPSG:25832, which provides a projected coordinate system suitable for metric distance and area calculations such as 100 m proximity analysis and geometry intersection.

4. System Architecture

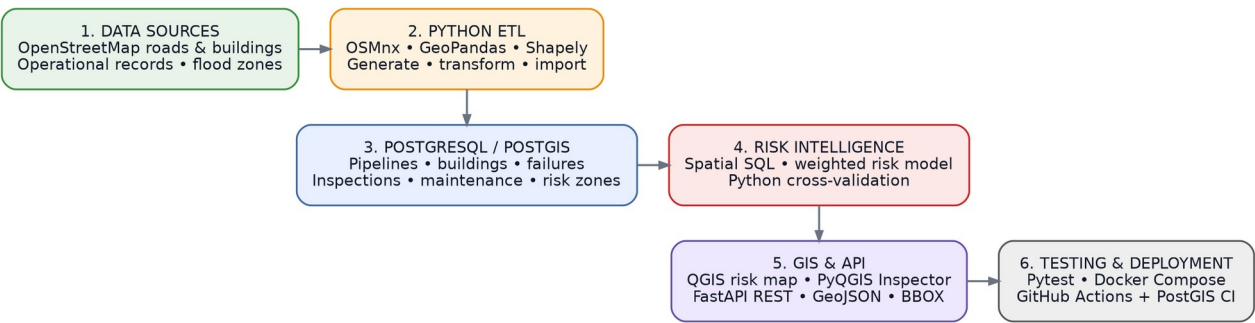


Figure 1. GeoAsset Intelligence system architecture.

The architecture is organized into six stages. Spatial and operational inputs are transformed through Python ETL, loaded into PostGIS, analyzed through SQL and Python validation, delivered through QGIS and FastAPI, and verified through automated testing and containerized deployment. This structure keeps analytical logic close to the spatial database while supporting multiple user-facing clients.

5. PostGIS Database Design

PostgreSQL/PostGIS is the authoritative data store. The schema separates physical assets, operational events, environmental context, and derived risk outputs. This supports clear relationships between a pipeline and its failures, inspections, and maintenance history while preserving spatial indexing and GIS compatibility.

Object	Purpose
pipelines	Pipeline asset geometry and core attributes
pipeline_failures	Failure events linked to pipeline assets
inspections	Inspection history and dates
maintenance	Maintenance activity linked to pipeline assets
buildings	Urban exposure context used for proximity analysis
risk_zones	Flood-risk polygons used for exposure analysis
pipeline_risk_metrics	Derived analytical metrics used by the risk model
pipeline_risk_assessment	Weighted risk score and risk class
pipeline_risk_map	Map-ready view consumed by QGIS

Spatial indexes support proximity and intersection operations. The database design enables the same derived outputs to be reused by QGIS, Python validation scripts, and the API instead of recalculating risk independently in each client.

6. Python ETL and Data Engineering

The ETL layer is implemented in Python and uses OSMnx, GeoPandas, Shapely, Pandas, SQLAlchemy, and Psycpg. The workflow downloads and processes OSM data, generates a synthetic water network from road geometry, creates operational records, imports data into PostGIS, and validates analytical outputs.

- Download and prepare OSM roads and building footprints.
- Generate 500 pipeline segments from realistic road geometry.
- Generate failure, inspection, and maintenance histories.
- Import spatial and non-spatial records into PostgreSQL/PostGIS.
- Read PostGIS risk views back into Python for independent verification.
- Use GeoPandas spatial joins and distance predicates to reproduce key exposure metrics.

A key engineering choice was to maintain independent SQL and Python implementations of the risk calculation. Agreement between the two provides stronger evidence that the result is correct than relying on a single code path.

7. Risk Assessment Methodology

Each pipeline receives a composite score from 0 to 100. The model is intentionally transparent: every point is attributable to an interpretable operational or spatial factor. Component contributions are capped so that a single extreme variable cannot dominate the entire score.

Risk factor	Cap / threshold	Maximum points
Asset age	50 years	25
Days since last inspection	365 days	10
Failure history	3 failures	30
Buildings within 100 m	10 buildings	15
Flood exposure	Exposure proportion	20
Total		100

The model places the highest weight on failure history, followed by age and flood exposure. Nearby development represents consequence/exposure, while inspection recency reflects information quality and maintenance attention. PostGIS calculates the production metrics using distance and intersection relationships; the Python risk engine reproduces the calculation for validation.

Risk Classification

Risk score	Class	Interpretation
0-30	Low	Routine monitoring
>30-60	Medium	Elevated attention
>60-80	High	Prioritized review / inspection
>80	Critical	Highest maintenance or investigation priority

Network Risk Distribution

Risk level	Pipelines	Average score
Low	210	21.31
Medium	260	42.85
High	29	64.93
Critical	1	87.46

Interpretation

Most assets fall in the Low or Medium classes, while only 30 pipelines are High/Critical. This creates a focused shortlist for inspection and maintenance prioritization rather than treating the entire network as equally urgent.

8. QGIS Analysis and Visualization

QGIS is used as the principal desktop GIS interface. The map combines risk-ranked pipeline segments with building footprints and failure points, allowing the analytical score to be interpreted in spatial context. Pipeline symbology is categorized into Low, Medium, High, and Critical classes, with higher-risk segments visually emphasized.

The map is not only a presentation product; it also acts as a verification environment. Analysts can compare high-risk lines against dense built-up areas, failure locations, and risk-zone exposure, and can use standard QGIS inspection tools to review individual attributes.

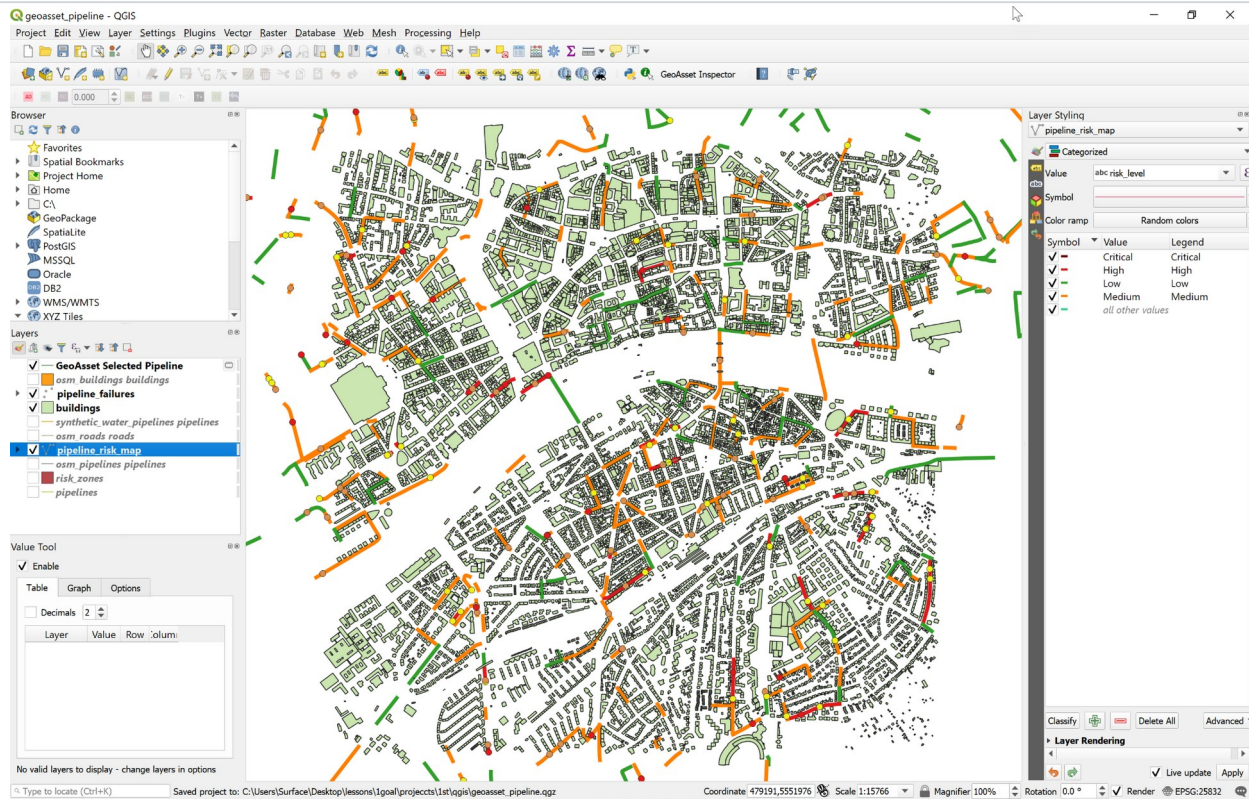


Figure 2. QGIS overview of the pipeline risk map, building context, failure points, and categorized risk symbology.

The visual distribution confirms that elevated risk is concentrated in a relatively small set of pipeline segments rather than spread uniformly across the network. This supports the prioritization objective of the project.

9. GeoAsset Inspector - PyQGIS Integration

To move beyond static visualization, the project includes a custom QGIS plugin named GeoAsset Inspector. The plugin accepts a pipeline ID, queries the FastAPI service, displays the asset and risk attributes inside QGIS, creates a temporary selected-pipeline layer, and zooms to the returned geometry.

The implementation deliberately uses lightweight QGIS/PyQt capabilities and Python standard-library HTTP access. It was tested with QGIS 3.12.2, which required careful attention to compatibility while keeping the plugin dependency-free inside QGIS.

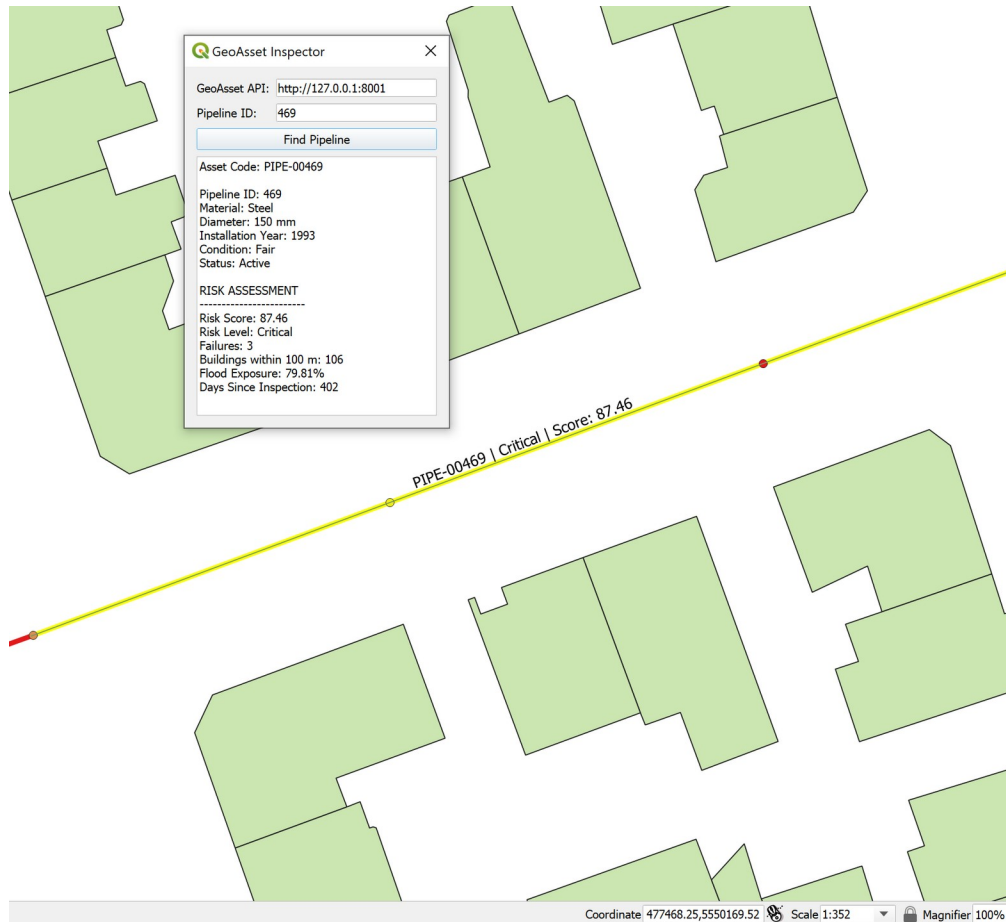


Figure 3. GeoAsset Inspector retrieving PIPE-00469 and displaying its Critical risk assessment directly in QGIS.

PIPE-00469 demonstrates the complete workflow. At the time of capture, the plugin reported a risk score of 87.46, risk level Critical, 3 failures, 106 buildings within 100 m, 79.81% flood exposure, and 402 days since the latest inspection. The pipeline geometry is simultaneously highlighted on the map.

Why the plugin matters

The plugin closes the loop between backend analytics and desktop GIS. QGIS is not merely reading a database layer; it is consuming a geospatial API and turning API responses into an operator-facing inspection workflow.

10. FastAPI Geospatial Service

FastAPI exposes pipeline assets, operational histories, risk results, statistics, and geometry as JSON or GeoJSON. This separates data access from the desktop GIS and makes the analytical outputs reusable by other clients. Spatial bounding-box queries provide an API-level pattern for map-window retrieval.

Endpoint group	Examples
Health	/health, /database-health
Assets	/pipelines, /pipelines/{pipeline_id}
Spatial	/pipelines/bbox, /pipelines/{pipeline_id}/geojson
Operations	/failures, /inspections, /maintenance by pipeline
Risk	/risk/high, /risk/high/geojson, /risk/summary
Statistics	/stats

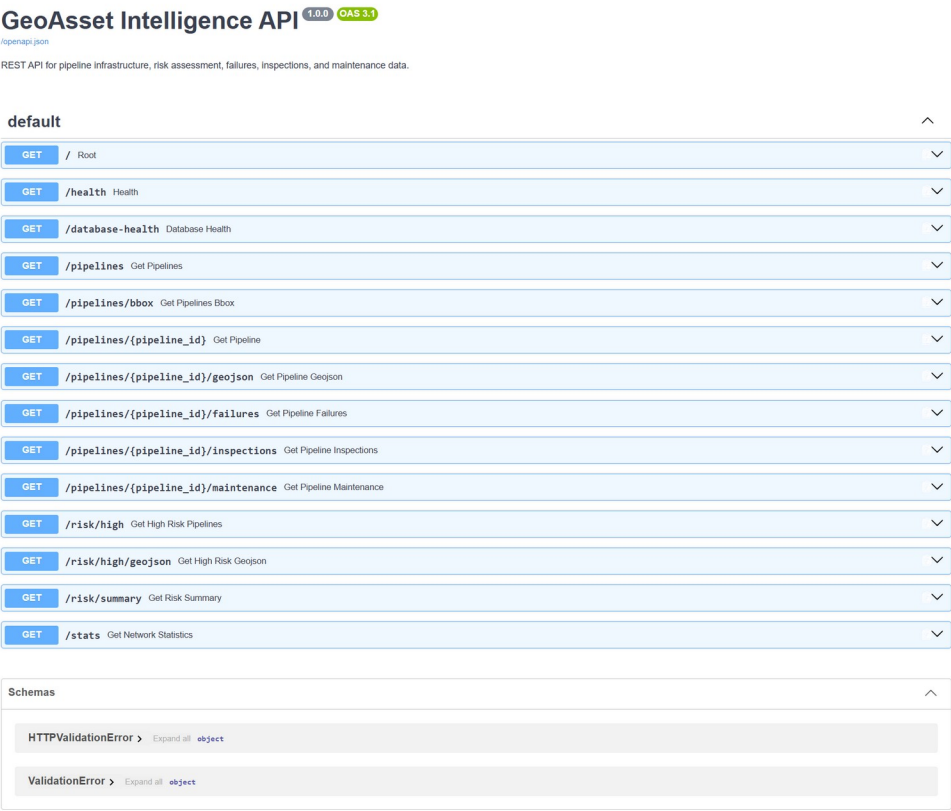


Figure 4. FastAPI Swagger interface showing the GeoAsset Intelligence REST endpoints.

GeoJSON responses can be loaded directly into GIS software, while the same endpoints support the GeoAsset Inspector plugin. Database credentials are supplied through environment variables rather than committed to the repository.

11. Testing, CI, and Containerized Deployment

The project includes both unit testing and database-backed integration testing with Pytest. The API tests exercise live PostGIS data when database credentials are available, while risk-calculator unit tests verify core scoring behavior independently of the database.

GitHub Actions runs a full integration workflow on each push or pull request to the main branch:

- Start a PostGIS service container.
- Create the schema and analytical SQL views.
- Load deterministic CI test data containing Low, Medium, High, and Critical examples.
- Run the complete Pytest suite against the database.
- Build the Docker image to verify reproducible packaging.

Locally, Docker Compose runs the FastAPI service while connecting to the PostGIS database through `host.docker.internal`. Database passwords remain outside version control and are passed as environment variables. This setup demonstrates a practical development deployment pattern without adding unnecessary cloud infrastructure.

12. Results

Metric	Result
Pipeline segments	500
Total network length	54.34 km
Average pipeline age	31.04 years
Average diameter	184.64 mm
Failure records	267
High-risk pipelines	29
Critical pipelines	1
High + Critical	30
Top-ranked asset	PIPE-00469
Top risk score	87.46

The system successfully identifies a small, interpretable high-priority subset of the network. PIPE-00469 is the strongest example: repeated failures, high surrounding building density,

substantial flood exposure, and an overdue inspection combine into the only Critical score in the dataset. The result is visible consistently in PostGIS, QGIS, the PyQGIS plugin, and the API.

Case Study: PIPE-00469

Attribute	Value
Material	Steel
Diameter	150 mm
Installation year	1993
Condition	Fair
Status	Active
Failure count	3
Buildings within 100 m	106
Flood exposure	79.81%
Risk score	87.46
Risk class	Critical

13. Discussion and Limitations

The strongest aspect of GeoAsset Intelligence is integration. The project demonstrates how geospatial data engineering, spatial SQL, desktop GIS, APIs, and software engineering practices can operate around one consistent analytical model. The PostGIS database is the shared analytical core, while Python and QGIS provide validation and user interaction rather than duplicating disconnected logic.

The risk model is intentionally explainable and suitable for a portfolio demonstration, but it is not a calibrated engineering failure model. The weights and caps are heuristic. A production utility implementation would require domain-specific calibration using historical failures, pipe material deterioration models, hydraulic context, soil conditions, consequence-of-failure estimates, and organization-specific inspection standards.

Other limitations include the use of synthetic pipeline and operational data, the absence of authentication/authorization in the API, no real-time sensor integration, and local rather than cloud deployment. These are conscious scope decisions: the project emphasizes the geospatial

engineering workflow rather than adding infrastructure that would not materially improve the portfolio objective.

14. Conclusion

GeoAsset Intelligence demonstrates a complete and reproducible infrastructure-risk workflow built around modern open-source geospatial technologies. Starting with realistic spatial context, the project creates and loads asset data, performs risk analysis in PostGIS, validates the results in Python, visualizes them in QGIS, exposes them through FastAPI, and verifies the system with automated PostGIS integration tests and Docker builds.

The project therefore shows practical capability across GIS development, spatial SQL, Python geospatial engineering, backend API development, QGIS automation, testing, and DevOps. Its most important outcome is not a single map or score, but a coherent system in which the same spatial intelligence can be queried, validated, visualized, and delivered through multiple interfaces.

Portfolio positioning

This project is directly relevant to GIS Developer, Python GIS, PostGIS / Spatial SQL, Geospatial Backend, Spatial Data Engineering, and utility/infrastructure GIS roles.

Repository

Source code, SQL, ETL scripts, QGIS project, plugin, tests, Docker configuration, CI workflow, and documentation: github.com/AisunZa/geoasset-intelligence-platform

Appendix A - Technology Stack

Area	Technologies
Desktop GIS	QGIS, PyQGIS
Spatial database	PostgreSQL 15, PostGIS
Python	Python 3.13, GeoPandas, Shapely, Pandas, OSMnx
Database access	SQLAlchemy, GeoAlchemy2, Psycopg
API	FastAPI, Uvicorn, JSON, GeoJSON
Testing	Pytest, FastAPI TestClient / HTTP integration
Deployment	Docker, Docker Compose
CI	GitHub Actions with PostGIS service
CRS	EPSG:25832

Appendix B - Repository Structure

```

geoasset-intelligence-platform/
├── api/                FastAPI application
├── database/           PostGIS schema, views, CI seed data
├── etl/                OSM, generation, import, risk scripts
├── qgis/               QGIS project
├── qgis_plugin/        GeoAsset Inspector plugin
├── test/               Unit and integration tests
├── docs/images/        Portfolio visuals
├── Dockerfile
├── docker-compose.yml
├── environment.yml
└── README.md

```